PunkBackup — Configuración del iPhone, Construcción Manual (Método alternativo)

Este es el camino alternativo. Casi todos deberían usar el **Manual de configuración del iPhone** principal — instala un archivo de Atajo ya armado en cerca de un minuto. Usa esta guía solo si prefieres construir cada acción a mano, o quieres entender exactamente qué hace cada paso.

Antes de empezar, ten a la mano

Mismo requisito que el manual principal: este sistema soporta varios dispositivos/personas (perfiles) en la misma PC, y **cada dispositivo necesita su propio perfil y su propio token**.

En tu PC: 1. Abre la app y presiona **Iniciar backup**. 2. En **"Configuración"**, click **"+ Agregar perfil"** y ponle un nombre que identifique ESTE dispositivo (ej. `iPhone de María`), y elige su carpeta destino cuando te la pida. 3. La app te muestra (y copia automáticamente) el **token** de ese perfil — solo sirve para ese dispositivo.

Anota estos datos:

Dato	Dónde lo ves en la app	Ejemplo
Dirección del servidor (igual para todos los perfiles)	Campo "Dirección"	`http:■/■/■[NOMBRE_DE_TU_PC].■local:■`
IP alternativa (por si la anterior no responde desde el iPhone)	Campo "IP alternativa"	`http:■/■/■192.■168.■1.■50:■8787`
Token de ESTE perfil	Botón "Copiar token" en la fila del perfil que acabas de crear	una cadena larga de letras/números

> Tu iPhone y tu PC deben estar conectados a la **misma red WiFi**.

Paso 1 — Crear el Atajo principal ("PunkBackup")

1. Abre la app **Atajos**.
2. Pestaña **Mis Atajos** → botón **+** → **Añadir Acción** (nuevo atajo).

3. Toca el nombre en la parte superior y cámbialo a: `PunkBackup`.
4. Agrega las siguientes acciones **en este orden** (busca cada una por nombre con la lupa):

1) Guardar la dirección del servidor y el token como variables

- Acción **Texto** → escribe la Dirección del servidor, ej. `http:■/■/■[NOMBRE_DE_TU_PC].■local:■8787`
- Acción **Establecer variable** → nombre `ServidorURL` → valor: el Texto anterior.
- Acción **Texto** → pega el Token **de este perfil/dispositivo** que copiaste de la app.
- Acción **Establecer variable** → nombre `Token` → valor: el Texto anterior.

> Estas 2 son las únicas acciones que vas a tener que editar si algún día renuevas el token de este perfil desde la app (botón "Renovar token").

2) Avisar al servidor que empieza una corrida de backup

- Acción **Obtener contenido de URL**:
- URL: `ServidorURL` + `/run/start` (usa "Combinar texto" o escribe la URL tocando la variable `ServidorURL` y agregando `/run/start` a continuación).
- Método: **POST**
- Encabezados (Headers): agrega uno → Clave `X-Backup-Token` → Valor: variable `Token`.
- Acción **Obtener valor de diccionario** → Clave: `run_id` → (aplicado al resultado del paso anterior).
- Acción **Establecer variable** → nombre `RunID` → valor: el resultado anterior.

3) Armar el barrido en bloques hacia atrás

`Buscar fotos` no tiene paginación integrada — pedirle todo de una vez directamente falla en una biblioteca grande (comprobado: sin `Limit`, el Atajo da error incluso solo contando resultados, sin subir nada). La solución es barrer la biblioteca **hacia atrás en bloques acotados**, empezando por las fotos más recientes, de a 50 a la vez, avanzando un límite de fecha móvil entre bloque y bloque — todo dentro de una sola corrida del Atajo.

- Acción **Texto** → escribe `50` (cuántos bloques de 50 barrer por corrida — ver la nota de ajuste al final de esta sección).
- **Establecer variable** → nómbrala `Repeticiones`.
- Acción **Fecha actual** (Current Date).
- Acción **Ajustar fecha** (Adjust Date) → **Sumar 1 día** a esa fecha (así el primer bloque no excluye nada — "antes de mañana" cubre todo lo que tienes hoy).
- **Establecer variable** → nómbrala `Limite` (tipo Fecha — déjalo como valor de Fecha real, no texto; el siguiente paso lo compara directamente contra la fecha de captura de cada foto).
- Acción **Repetir** (la versión simple "Repeat X times", **no** "Repetir con cada elemento" — este es el loop externo) → como cantidad, inserta la variable **Repeticiones** en vez de escribir un número fijo.

Todo lo que sigue, hasta el final de la sección 4, va **dentro** de este `Repetir` externo:

- Acción **Buscar fotos** (o "Filtrar fotos" según tu versión de iOS):
- Agrega un filtro: **Fecha de captura** → **es antes de** (is before) → inserta la variable **Limite**.

- Ordenar por: **Fecha de captura**. Orden: **Más reciente primero** (Latest First) — es lo opuesto a lo natural, y es justo lo que hace que el barrido empiece por tus fotos más nuevas y avance hacia atrás.
- **Limit**: actívalo, ponlo en **50**. Esto es obligatorio, no opcional — `Buscar fotos` sin `Limit` falla en una biblioteca grande.
- Acción **Contar** (Count) → **Items** sobre el resultado de `Buscar fotos`.
- Acción **Si** (If): condición = el resultado de `Count` **es** (is) `0` (esto se cumple cuando el barrido ya pasó tu foto más vieja — no queda nada por revisar).
- Dentro del "Si": acción **Stop This Shortcut**.
- No hace falta nada en "Si no" — si el conteo no es 0, la ejecución simplemente sigue después del "End If" hacia la sección 4.

4) Recorrer cada foto/video del bloque actual: preguntar primero, subir solo si falta

Acción **Repetir con cada elemento** (Repeat with Each) sobre el resultado de `Buscar fotos` de la sección 3. Dentro de este bloque "Repetir" interno:

Paso a.0) Acción **Establecer variable**, como la primera acción de este bloque:

- Valor: toca "Elemento de repetición" (Repeat Item) y elige el atributo **Fecha de captura** (Date Taken) — déjalo como Fecha cruda, sin pasarlo por Formatear fecha aquí.
- Nombra la variable `UltimaFecha`. Esto captura la fecha exacta del elemento actual; al terminar el loop, queda con la fecha del elemento MÁS VIEJO de ese bloque (porque está ordenado de más nuevo a más viejo) — los pasos finales de la sección 4 la usan para mover `Limite` de cara al siguiente bloque.

Paso a) Acción **Formatear fecha**:

- Fecha: toca el "Elemento de repetición" (Repeat Item) y elige el atributo **Fecha de captura** (Date Taken).
- Formato: **Personalizado** → escribe: `yyyy-MM-dd'T'HH:mm:ss`
- Establece esto como variable `TomadaEn` (acción **Establecer variable**).
- **Chip delicado, verifícalo después de guardar**: si más adelante agregas o mueves cualquier acción DENTRO de este mismo bloque "Repetir" (por ejemplo el paso a.0 de arriba), Shortcuts puede reconfigurar en silencio este chip para que apunte a otro atributo (como "Nombre") en vez de "Fecha de captura" — sin marcarlo en rojo, sin ningún aviso visible. El síntoma es que, semanas después, TODAS las fotos nuevas terminan en la misma carpeta (la del mes actual) en vez de en su mes real. Si sospechas esto, toca el chip dentro de "Formatear fecha" y confirma que dice **Fecha de captura**, no Nombre ni ningún otro atributo.

Paso a.2) Acción **Texto** — arma el nombre completo, **con extensión** (paso importante: el atributo "Nombre de archivo" de Shortcuts, por sí solo, **no incluye la extensión** — sin este paso vas a terminar con archivos como `IMG_1234` en vez de `IMG_1234.HEIC`):

- En el campo de texto, inserta: **Elemento de repetición** → elige el atributo **Nombre de archivo** → escribe un punto `.` (sin espacios) → inserta **Elemento de repetición** otra vez → elige el atributo **Extensión de archivo** (File Extension).

- Debe quedar algo como: `[Nombre de archivo].[Extensión de archivo]`
- Establece esto como variable `NombreArchivo` (acción **Establecer variable**).

Paso b) Acción **Obtener contenido de URL** — el chequeo liviano, SIN el archivo:

- URL: `ServidorURL` + `/check`
- Método: **POST**
- Tipo de solicitud: **Formulario** (Form)
- Campos del formulario:
  - `filename` → valor: variable **NombreArchivo** (la del paso a.2 — NO uses el atributo "Nombre de archivo" directo, le falta la extensión).
  - `taken_at` → valor: variable `TomadaEn`.
- No hace falta mandar el tamaño del archivo — Shortcuts no tiene forma confiable de dar el tamaño en bytes puros (siempre da algo como "1,2 MB"), así que el chequeo solo usa nombre + fecha. La verificación exacta por contenido pasa después, en el `/upload`.
- Encabezados: `X-Backup-Token` → variable `Token`.
- Acción **Obtener valor de diccionario** → clave `missing` → sobre ese resultado.

Paso c) Acción **Si** (If): condición = el valor anterior **has any value** ("tiene algún valor" — así es como el servidor te dice "falta, súbela"; si ya está respaldada, el servidor no manda el campo `missing` en absoluto, así que esta condición da falso automáticamente y se salta el "Si" — no necesitas elegir "is equal to" ni escribir `false` en ningún lado, "has any value" ya es la opción correcta):

> **Paso c.0) Si** (primera acción dentro del "Si" del paso c — decide si este ítem necesita Encode Media ANTES de intentar subirlo, no como reintento después de que falle):
>
> > Condición: toca **Elemento de repetición** → atributo **Media Type** → `is` → `Video`. (Si tu versión de iOS no ofrece "Media Type" en la lista de atributos del Elemento de repetición, usa en su lugar: **Elemento de repetición** → **File Extension** `is` `MOV`, y con **+** agrega más condiciones unidas por **Or**: `is` `MP4`, `Or` → `is` `M4V`.)
> >
> > **Rama Video** (dentro de este "Si"):
> >
> > - Acción **Encode Media** → Ítem: inserta **Elemento de repetición** — una sola vez, sin volver a tocarlo (mismo cuidado de siempre con este chip).
> > - **Size**: `Passthrough` (no reduce calidad ni resolución — solo fuerza a Shortcuts a leer el archivo completo, que es justo lo que falla para algunos videos — ver la nota de más abajo).
> > - **Set Variable** → nómbrala `ItemFile` → valor: el resultado de Encode Media de arriba.
> >
> > **Rama "Si no"** (foto, no video):
> >
> > - **Set Variable** → nómbrala `ItemFile` → valor: inserta **Elemento de repetición** directo — una sola vez, sin volver a tocarlo.
>
> **Paso c.1) Obtener contenido de URL** (dentro del "Si" del paso c, después del c.0 — este es ahora el único intento de subida):

    - URL: toca el campo e inserta, EN ESTE ORDEN, dentro del mismo campo de texto: chip **ServidorURL** → escribe `/upload?filename=` → inserta el chip de la variable **NombreArchivo** (la misma del paso a.2 — NO el atributo "Nombre de archivo" suelto, le falta la extensión) → escribe `&taken_at=` → chip **TomadaEn** → escribe `&run_id=` → chip **RunID**. Insertando los chips directamente en el campo de URL (no armando el texto aparte con "Combinar texto"), Shortcuts los codifica automáticamente para que la URL quede válida.
    - Método: **POST**
    - **Request Body** → cámbialo a **File** (ya NO es Formulario).
    - En el campo de valor que aparece, inserta la variable **`ItemFile`** (del paso c.0) — NO Elemento de repetición, NO Encode Media directo. Si este chip alguna vez aparece resaltado en rojo/"roto" en el editor — puede pasar después de editar otras acciones más arriba en el mismo loop — bórralo y vuelve a insertarlo igual que antes; una referencia rota aquí sube un archivo vacío (0 bytes) en vez de dar un error visible.
    - Encabezados: `X-Backup-Token` → variable `Token`.

    (No hace falta ninguna acción en el "Si no" del "Si" del paso c — si ya estaba respaldada, simplemente no se hace nada y se sigue con la siguiente foto).

> **Por qué Encode Media corre por adelantado en vez de como reintento**: se confirmaron, en dispositivo real, dos problemas distintos con la misma raíz. Primero (2026-09-16): un video importado de WhatsApp/Mensajes/otras apps a veces llega al servidor como 0 bytes — Shortcuts no logra leer sus datos reales de la librería de Fotos directamente. Segundo, y la razón de este rediseño (2026-10-23): algunos videos hacen que el intento directo falle por completo, en el propio editor de Shortcuts, con la alerta `WFPhotoMediaContentItem produced no file representation for type public.mpeg-4` — un fallo al convertir el ítem de Fotos en bytes de archivo, ANTES de que se envíe cualquier petición HTTP. El diseño anterior (intentar el ítem crudo primero, reintentar con Encode Media solo si la respuesta del servidor traía un campo `detail`) nunca podía atrapar esta segunda falla, porque no existe ninguna respuesta del servidor que revisar — la acción de subida ni siquiera llega a ejecutarse. Correr "Encode Media" con `Size: Passthrough` para todo video, sin condición (filtrado solo por el `/check` del paso b, así que un video ya respaldado nunca paga este costo), evita ambos problemas a la vez. Confirmado sin pérdida real de calidad (comparación con `ffprobe`: mismo códec, misma resolución, mismo bitrate — ver PLAN.md §5.1) y confirmado en producción (2026-10-23): una corrida real subió videos de hasta 69.8 MB sin ningún rechazo de 0 bytes y sin ningún error de conversión. Ver AGENTS.md §5, lección 28, para el análisis completo — incluyendo por qué esto no crea riesgo de perder un video aunque ya no exista reintento dentro de la misma corrida (el servidor nunca registra una subida fallida o parcial como respaldada, así que el `/check` la vuelve a detectar en la siguiente corrida).

Todavía dentro del "Repetir con cada elemento" **interno**, después de su propio "End Repeat" pero **antes** del "End Repeat" externo de la sección 3 — estas dos acciones avanzan el barrido al siguiente bloque:

- Acción **Ajustar fecha** (Adjust Date) → **Restar 1 minuto** a **UltimaFecha** (un pequeño colchón de seguridad, para que una foto con el mismo timestamp exacto que el límite del bloque — por ejemplo fotos en ráfaga — nunca se salte en silencio; en el peor caso se vuelve a revisar de más, lo cual es inofensivo).

- Acción **Establecer variable** → variable: **Limite** (elige la variable `Limite` ya existente en la lista, no crees una nueva con el mismo nombre) → valor: el resultado del `Ajustar fecha` de arriba.

> **Ajustando `Repeticiones`**: comprobado funcionando con `Repeticiones` hasta 50 (≈2.500 fotos revisadas en una sola corrida) en pruebas reales; `Repeticiones = 180` (≈9.000 fotos) también fue confirmado sin que el Atajo mismo falle ni se cuelgue. No asumas que un bloque ya respaldado es rápido solo porque no transfiere ningún archivo — medido directamente de logs de actividad reales, un chequeo de "ya respaldado" puede tardar igual varios segundos por elemento, no un ida-y-vuelta casi instantáneo. Ver la sección "¿Qué tan rápido es un respaldo, en la práctica?" del Manual de solución de problemas para los números medidos (incluye una tabla de tiempo por `Repeticiones`) y por qué una corrida que está mayormente re-revisando contenido ya respaldado puede tardar más que una que sube la misma cantidad de elementos totalmente nuevos.

> **Limitación importante, no obvia**: `Limite` siempre se reinicia a "mañana" al empezar cada corrida — no hay memoria de dónde quedó la corrida *anterior*. Si una corrida termina porque se acabaron sus `Repeticiones` (y no porque encontró un bloque vacío y llegó a tu foto más vieja), **volver a correrla con el mismo `Repeticiones` no avanza nada más** — vuelve a barrer exactamente las mismas fotos más recientes y se detiene exactamente en el mismo punto, siempre. Para de verdad llegar a fotos más viejas, hay que **subir `Repeticiones`**, no solo volver a correr el atajo. Para un respaldo inicial completo de una biblioteca grande, pon `Repeticiones` lo suficientemente alto para cubrir toda tu biblioteca en una sola corrida (aproximadamente `tu total de fotos ÷ 50`, redondeado hacia arriba — ej. ~180 para una biblioteca de 9.000 fotos) en vez de un número chico que piensas repetir — no está confirmado que iOS aguante tantos bloques seguidos con números muy altos, así que súbelo de a poco y revisa el log de actividad de la app en la PC para confirmar que va avanzando bien.

5) Cerrar la corrida y mostrarte el resultado

- Acción **Obtener contenido de URL**:
- URL: `ServidorURL` + `/run/finish`
- Método: **POST**, Formulario con campo `run_id` = variable `RunID`.
- Encabezado `X-Backup-Token` → variable `Token`.
- Acciones **Obtener valor de diccionario** (una por cada dato: `files_new`, `files_skipped`, `files_conflict`) sobre ese resultado.
- Acción **Combinar texto** para armar un mensaje, ej.: `Backup completo \nNuevos: [files_new]\nYa existían: [files_skipped]\nConflictos: [files_conflict]`
- Acción **Mostrar notificación** (o **Mostrar resultado**) con ese texto.

Guarda el atajo (listo, ya puedes tocarlo manualmente para probarlo). (Ver la nota "Ajustando `Repeticiones`" en la sección 3 para saber cuánto tarda una corrida en una biblioteca grande, y cómo dimensionar el barrido para tu primer respaldo completo.)

Paso 2 — Atajo corto para consultar el estado ("Estado del Backup")

Opcional pero recomendado — te deja ver el estado sin correr un backup completo.

1. Crea un nuevo atajo llamado `Estado del Backup`.
2. Repite las 2 acciones de "Texto" + "Establecer variable" del Paso 1.1 (`ServidorURL`, `Token`).
3. Acción **Obtener contenido de URL**: URL = `ServidorURL` + `/status`, Método **GET**, encabezado `X-Backup-Token` → `Token`.
4. Acciones **Obtener valor de diccionario** para `last_backup_at` y `total_files_backed_up`.
5. Acción **Mostrar notificación** con esos valores.

Paso 3 — Automatizar

Igual que el paso de automatización del manual principal — una vez que tu atajo `PunkBackup` construido a mano funcione, ve a la sección "Automatizar: que se dispare solo al llegar a casa" del **Manual de configuración del iPhone** principal y síguela igual, sin importar cómo construiste el Atajo.

Notas

- El sistema **nunca borra ni modifica nada en tu iPhone** — solo lee fotos y las sube.
- La verdad de "qué ya está respaldado" vive **solo en la carpeta destino de la PC**, nunca en el iPhone. Si en la PC cambias la carpeta/USB de tu perfil, el atajo automáticamente vuelve a enviar lo que falte en la carpeta nueva — no necesitas hacer nada distinto en el iPhone.
- Los nombres exactos de las acciones pueden variar levemente entre versiones de iOS; si no encuentras una acción con el nombre exacto, busca por una palabra clave (ej. "diccionario", "URL", "repetir").

Si también usas el backup por cable

No tienes que cambiar nada en el Atajo. La primera pasada después de un backup por cable vuelve a enviar los archivos que no son de cámara y tarda más, pero no se guardan dos veces. A partir de la segunda pasada ya no se piden.